Aprendizaje de una Red Neuronal Simple

Descenso por Gradiente paso a paso

Recurso Pedagógico

2026-03-16

Table of contents

Introducción **2**

1. Arquitectura de la Red Neuronal **2**
1.1 Estructura ($2 \to 1$) . . . 2
1.2 Ecuación de la neurona . . . 3

2. Función de Pérdida **3**
2.1 Error cuadrático medio (MSE) . . . 3

3. Cálculo del Gradiente **4**
3.1 Regla de la cadena . . . 4
3.2 Grafo computacional . . . 4
Lectura del grafo: retropropagación paso a paso . . . 4
Resumen: ¿cuándo se multiplica y cuándo se copia? . . . 5
3.3 Cada término del gradiente . . . 6

4. Actualización de Pesos — Descenso por Gradiente **6**
4.1 Regla de actualización . . . 6

5. Implementación en Python **7**
5.1 Configuración inicial . . . 7
5.2 Funciones de la red neuronal . . . 7
5.3 Bucle de entrenamiento . . . 8
5.4 Verificación de las predicciones finales . . . 10

6. Visualización del Proceso de Aprendizaje **11**
6.1 Evolución del costo . . . 11
6.2 Trayectoria de los pesos en el espacio 3D . . . 11
6.3 Mapa de contorno de la función de costo (w1 vs w2) . . . 13
6.4 Paso a paso interactivo (detalle de cada iteración) . . . 15
6.5 Evolución de los pesos individuales . . . 15
6.6 Superficie de activación aprendida . . . 16

7. Flujo de Entrenamiento Completo **18**

8. Resumen Conceptual **20**
¿Cómo aprende una red neuronal? 20
Analogía geográfica 20

9. Ejemplo adicional: grafo computacional con bifurcaciones **21**
9.1 La función y sus variables intermedias 21
9.2 El grafo computacional (Forward Pass) 22
9.3 Derivadas locales de cada nodo 22
9.4 Retropropagación paso a paso (Backward Pass) 23
9.5 La eficiencia del grafo computacional 24
9.6 Resumen: las dos reglas del grafo 24

Referencias y Lecturas Adicionales **25**

Introducción

Este documento es un recurso pedagógico interactivo para comprender cómo **aprende una red neuronal artificial simple**. Utilizaremos el problema de la **compuerta lógica AND** como caso de estudio, el cual tiene la siguiente tabla de verdad:

Entrada x_1	Entrada x_2	Salida deseada y
0	0	0
0	1	0
1	0	0
1	1	1

La red neuronal debe **aprender** estos patrones ajustando sus pesos mediante el algoritmo de **Descenso por Gradiente**.

---

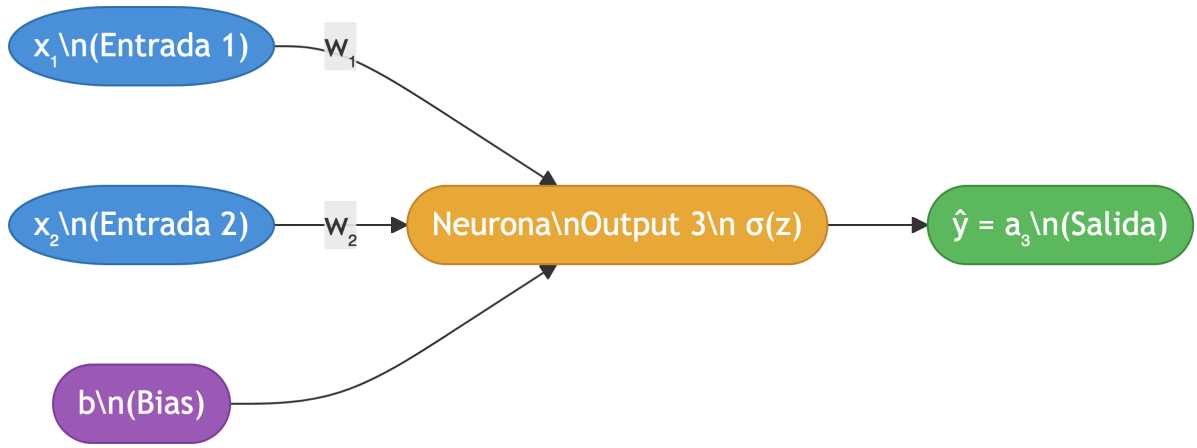
1. Arquitectura de la Red Neuronal

1.1 Estructura (2 → 1)

La red más simple posible tiene **dos entradas** y **una neurona de salida**:

```
flowchart LR
    X1(["x \n(Entrada 1)"]):::input
    X2(["x \n(Entrada 2)"]):::input
    N(["Neurona\nOutput 3\n  (z)"]):::neuron
    OUT(["ŷ = a \n(Salida)"]):::output

    X1 -- "w " --> N
    X2 -- "w " --> N
    B(["b\n(Bias)"]):::bias --> N
    N --> OUT
```

```
    classDef input fill:#4A90D9,stroke:#2C6FAC,color:white,rx:50
    classDef neuron fill:#E8A838,stroke:#C4842A,color:white
    classDef output fill:#5CB85C,stroke:#3D8B3D,color:white
    classDef bias fill:#9B59B6,stroke:#7D3C98,color:white
```



1.2 Ecuación de la neurona

La neurona realiza dos operaciones:

Paso 1 — Suma ponderada (pre-activación):

$$z = b + w_1 \cdot x_1 + w_2 \cdot x_2$$

Paso 2 — Función de activación sigmoide:

$$\hat{y} = a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

La función sigmoide comprime cualquier valor real al intervalo $(0, 1)$, lo que la hace ideal para salidas que representan probabilidades.

---

2. Función de Pérdida

2.1 Error cuadrático medio (MSE)

Para medir qué tan lejos está la predicción $\hat{y}$ del valor real y, usamos el **Error Cuadrático**:

$$C = (a - y)^2$$

El **costo total** sobre todos los ejemplos de entrenamiento es la suma:

$$C_{total} = \sum_{i=1}^{N} (a_i - y_i)^2$$

Intuición: Si $a = y$ (predicción perfecta), el costo es 0. Cuanto mayor sea el error, mayor será el costo. El objetivo del entrenamiento es **minimizar** esta función.

---

3. Cálculo del Gradiente

3.1 Regla de la cadena

Para actualizar los pesos necesitamos saber **en qué dirección decrece el error**. Usamos la **regla de la cadena** del cálculo diferencial:

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$\frac{\partial C}{\partial b} = \frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b}$$

3.2 Grafo computacional

El siguiente diagrama muestra el **grafo computacional completo** de la red neuronal: las flechas sólidas representan el *forward pass* y las flechas discontinuas el *backpropagation* (cálculo de gradientes mediante la regla de la cadena).

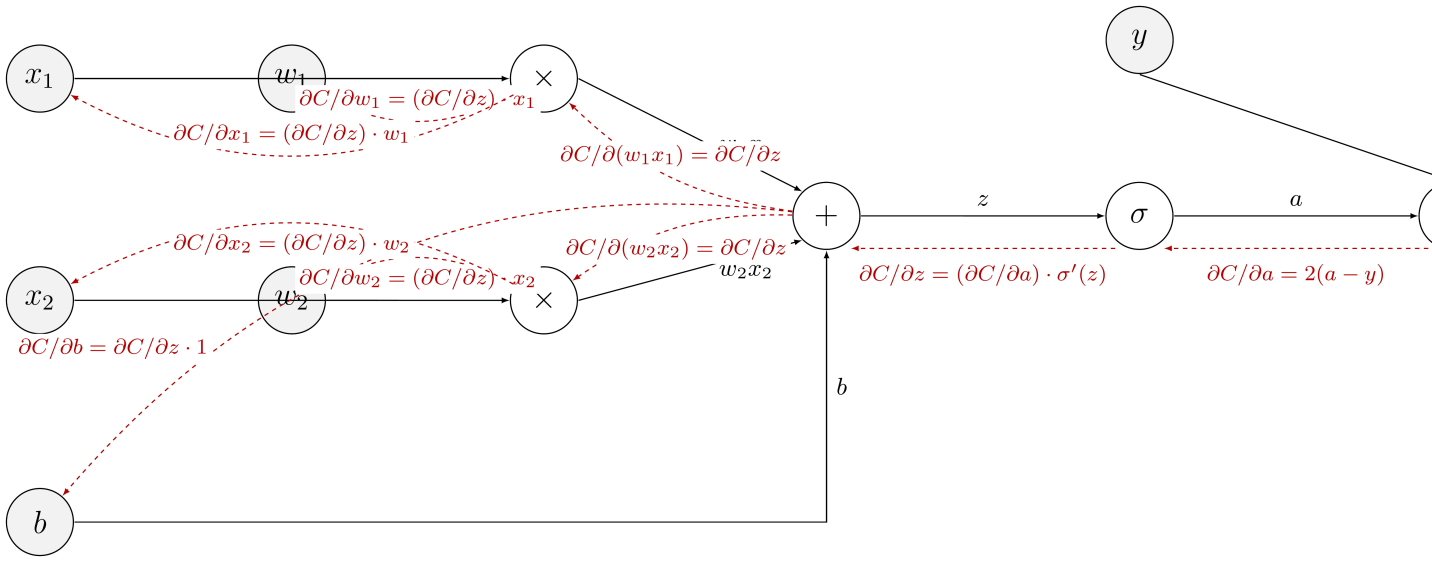


Figure 1: Grafo computacional completo: red neuronal 2 entradas → 1 salida

Lectura del grafo: retropropagación paso a paso

La red hace operaciones en secuencia (*forward pass*, flechas sólidas). Para aprender, necesita responder: **"si muevo un peso un poquito, ¿cuánto cambia el error?"**. La regla de la cadena permite responder eso recorriendo el grafo **al revés** (flechas discontinuas rojas).

Paso 1 — Nodo C (el costo):

$$C = (a - y)^2 \quad \Longrightarrow \quad \frac{\partial C}{\partial a} = 2(a - y)$$

Si movemos a un poquito, el costo cambia proporcionalmente a $2(a - y)$.

Paso 2 — Nodo σ (la sigmoide):

$$a = \sigma(z) \quad \Longrightarrow \quad \frac{\partial C}{\partial z} = \frac{\partial C}{\partial a} \cdot \sigma'(z)$$

Queremos cuánto cambia C al mover z, pero z afecta a C **a través** de a. La regla de la cadena dice: el efecto total es el **producto** de cada eslabón. Ya conocemos $\partial C/\partial a$ del paso anterior, así que solo multiplicamos por la derivada local $\sigma'(z)$.

Paso 3 — Nodo + (la suma):

$$z = w_1 x_1 + w_2 x_2 + b$$

La suma tiene tres entradas. La derivada de una suma respecto a cualquiera de sus términos es siempre 1:

$$\frac{\partial z}{\partial (w_1 x_1)} = 1, \qquad \frac{\partial z}{\partial (w_2 x_2)} = 1, \qquad \frac{\partial z}{\partial b} = 1$$

Entonces el gradiente simplemente se **copia idéntico** a cada rama:

$$\frac{\partial C}{\partial (w_1 x_1)} = \frac{\partial C}{\partial z} \cdot 1 = \frac{\partial C}{\partial z}$$

Paso 4 — Nodos $\times$ (la multiplicación):

$$w_1 x_1 = w_1 \cdot x_1 \quad \Longrightarrow \quad \frac{\partial (w_1 x_1)}{\partial w_1} = x_1, \qquad \frac{\partial (w_1 x_1)}{\partial x_1} = w_1$$

La derivada de un producto respecto a un factor es **el otro factor**. Aplicando la regla de la cadena una vez más:

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial z} \cdot x_1 \qquad \frac{\partial C}{\partial w_2} = \frac{\partial C}{\partial z} \cdot x_2$$

Resumen: ¿cuándo se multiplica y cuándo se copia?

Tipo de nodo	¿Qué hace con el gradiente?	¿Por qué?
Cadena (un camino)	**Multiplica** los gradientes	Regla de la cadena: el efecto de A a través de B es el producto
Suma (+)	**Copia** el gradiente a cada rama	Porque $\partial(a+b)/\partial a = 1$ y $\partial(a+b)/\partial b = 1$
Multiplicación ($\times$)	**Multiplica** por el otro factor	Porque $\partial(ab)/\partial a = b$ y $\partial(ab)/\partial b = a$

Intuición: La regla de la cadena siempre **multiplica** a lo largo de un camino. El nodo + produce factor 1, así que parece que "no hace nada" — simplemente pasa el gradiente tal cual. Mientras que el nodo $\times$ produce factores que son los valores de las otras entradas.

3.3 Cada término del gradiente

Término	Fórmula	Significado
$\frac{\partial C}{\partial a}$	$2(a-y)$	Cuánto cambia el error respecto a la salida
$\frac{\partial a}{\partial z}$	$\sigma'(z) = \frac{e^z}{(1+e^z)^2}$	Pendiente de la activación sigmoide
$\frac{\partial z}{\partial w_i}$	x_i	La entrada correspondiente
$\frac{\partial z}{\partial b}$	1	El bias siempre tiene derivada 1

Por lo tanto:

$$\frac{\partial C}{\partial w_i} = 2(a-y) \cdot \sigma'(z) \cdot x_i$$

$$\frac{\partial C}{\partial b} = 2(a-y) \cdot \sigma'(z) \cdot 1$$

---

4. Actualización de Pesos — Descenso por Gradiente

4.1 Regla de actualización

Una vez calculado el gradiente, los pesos se actualizan en la **dirección opuesta** al gradiente (para bajar la función de costo):

$$w_{\text{nuevo}} = w_{\text{actual}} - k \cdot \frac{\partial C}{\partial w}$$

$$b_{\text{nuevo}} = b_{\text{actual}} - k \cdot \frac{\partial C}{\partial b}$$

Donde k es la **tasa de aprendizaje** (*learning rate*), que controla el tamaño del paso.

Importante: Los gradientes se promedian sobre todos los ejemplos de entrenamiento antes de actualizar:

$$\Delta w_i = \frac{1}{N} \sum_{j=1}^{N} \delta w_i^{(j)}$$

---

5. Implementación en Python

5.1 Configuración inicial

```
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')

#   Semilla para reproducibilidad
np.random.seed(42)

#   Datos: compuerta AND
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]], dtype=float)

y = np.array([0, 0, 0, 1], dtype=float)

print("Tabla de entrenamiento (AND gate):")
print("x1  x2  |  y")
print("-" * 14)
for xi, yi in zip(X, y):
    print(f" {int(xi[0])}   {int(xi[1])}  |  {int(yi)}")
```

```
Tabla de entrenamiento (AND gate):
x1  x2  |  y
--------------
 0   0  |  0
 0   1  |  0
 1   0  |  0
 1   1  |  1
```

5.2 Funciones de la red neuronal

```
def sigmoid(z):
    """Función de activación sigmoide."""
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_deriv(z):
    """Derivada de la sigmoide:  '(z) =  (z)·(1- (z))."""
    s = sigmoid(z)
    return s * (1.0 - s)

def forward_pass(X, b, w1, w2):
```

```
    """
    Paso hacia adelante (forward pass).
    Retorna z (pre-activación) y a (activación).
    """
    z = b + w1 * X[:, 0] + w2 * X[:, 1]
    a = sigmoid(z)
    return z, a

def compute_cost(a, y):
    """Costo total: suma de errores cuadráticos."""
    return np.sum((a - y) ** 2)

def compute_gradients(X, y, z, a):
    """
    Calcula los gradientes para b, w1 y w2.
    Retorna el promedio sobre todos los ejemplos.
    """
    N = len(y)
    dC_da = 2 * (a - y)           #  C/ a
    da_dz = sigmoid_deriv(z)      #  a/ z

    delta = dC_da * da_dz         # término común

    grad_b  = np.mean(delta * 1)         #  C/ b  ( z/ b = 1)
    grad_w1 = np.mean(delta * X[:, 0])   #  C/ w1 ( z/ w1 = x1)
    grad_w2 = np.mean(delta * X[:, 1])   #  C/ w2 ( z/ w2 = x2)

    return grad_b, grad_w1, grad_w2

print(" Funciones definidas correctamente")
print(f"   sigmoid(0)   = {sigmoid(0):.4f}  (debe ser 0.5)")
print(f"   sigmoid'(0)  = {sigmoid_deriv(0):.4f}  (debe ser 0.25)")
```

```
 Funciones definidas correctamente
   sigmoid(0)   = 0.5000  (debe ser 0.5)
   sigmoid'(0)  = 0.2500  (debe ser 0.25)
```

5.3 Bucle de entrenamiento

```
#   Parámetros iniciales (igual que en las imágenes)
b_init,  w1_init,  w2_init  = 1.0, -2.0, 3.0
learning_rate = 10
n_steps = 50

#   Entrenamiento
b, w1, w2 = b_init, w1_init, w2_init
```

```python
history = []   # guardamos el estado en cada paso

for step in range(n_steps + 1):
    # 1. Forward pass
    z, a = forward_pass(X, b, w1, w2)
    cost = compute_cost(a, y)

    # 2. Guardar estado
    history.append({
        'step'    : step,
        'b'       : b,
        'w1'      : w1,
        'w2'      : w2,
        'cost'    : cost,
        'outputs' : a.copy(),
    })

    if step == n_steps:
        break   # no actualizar en el último paso

    # 3. Backward pass - gradientes
    grad_b, grad_w1, grad_w2 = compute_gradients(X, y, z, a)

    # 4. Actualización de pesos
    b  -= learning_rate * grad_b
    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2

#   Mostrar resumen
print(f"{'Paso':>5} | {'b':>8} | {'w1':>8} | {'w2':>8} | {'Costo':>8}")
print("-" * 50)
for h in history[:16]:  # mostrar primeros 16 pasos
    print(f"{h['step']:>5} | {h['b']:>8.3f} | {h['w1']:>8.3f} | {h['w2']:>8.3f} | {h['cost']:>8

print(f"\n{'...' :>5}")
final = history[-1]
print(f"\n  Pesos finales (paso {final['step']}):")
print(f"   b  = {final['b']:.4f}")
print(f"   w1 = {final['w1']:.4f}")
print(f"   w2 = {final['w2']:.4f}")
print(f"   Costo final = {final['cost']:.6f}")
```

```
 Paso |        b |       w1 |       w2 |    Costo
--------------------------------------------------
    0 |    1.000 |   -2.000 |    3.000 |   1.5853
    1 |   -0.007 |   -2.202 |    2.976 |   1.2632
    2 |   -0.552 |   -1.902 |    3.098 |   1.1184
```

```
   3 |   -0.928 |   -1.543 |    3.175 |   1.0137
   4 |   -1.268 |   -1.204 |    3.150 |   0.9208
   5 |   -1.606 |   -0.856 |    3.028 |   0.8144
   6 |   -1.963 |   -0.466 |    2.816 |   0.6769
   7 |   -2.307 |   -0.009 |    2.568 |   0.5266
   8 |   -2.537 |    0.492 |    2.412 |   0.4057
   9 |   -2.709 |    0.929 |    2.323 |   0.3235
  10 |   -2.877 |    1.267 |    2.263 |   0.2711
  11 |   -3.021 |    1.537 |    2.248 |   0.2371
  12 |   -3.167 |    1.742 |    2.251 |   0.2136
  13 |   -3.296 |    1.913 |    2.282 |   0.1960
  14 |   -3.426 |    2.049 |    2.319 |   0.1820
  15 |   -3.542 |    2.170 |    2.370 |   0.1702

 ...

 Pesos finales (paso 50):
  b  = -5.7867
  w1 = 3.7875
  w2 = 3.7879
  Costo final = 0.048993
```

5.4 Verificación de las predicciones finales

```python
print("\n  Predicciones de la red entrenada:\n")
print(f"{'x1':>4} {'x2':>4} | {'y real':>8} | {'ŷ (prob)':>10} | {'ŷ (clase)':>10} | {' / ':>5}"
print("-" * 55)

b_f, w1_f, w2_f = final['b'], final['w1'], final['w2']
_, a_final = forward_pass(X, b_f, w1_f, w2_f)

for i, (xi, yi) in enumerate(zip(X, y)):
    pred_prob  = a_final[i]
    pred_class = int(pred_prob >= 0.5)
    correct    = " " if pred_class == int(yi) else " "
    print(f"{int(xi[0]):>4} {int(xi[1]):>4} | {int(yi):>8} | {pred_prob:>10.4f} | {pred_class:>
```

```
 Predicciones de la red entrenada:

  x1   x2 |   y real |   ŷ (prob) |  ŷ (clase) |    /
-------------------------------------------------------
   0    0 |        0 |     0.0031 |          0 |
   0    1 |        0 |     0.1193 |          0 |
   1    0 |        0 |     0.1193 |          0 |
   1    1 |        1 |     0.8568 |          1 |
```

---

6. Visualización del Proceso de Aprendizaje

6.1 Evolución del costo

```
costs = [h['cost'] for h in history]
steps = [h['step'] for h in history]

fig_cost = go.Figure()

fig_cost.add_trace(go.Scatter(
    x=steps, y=costs,
    mode='lines+markers',
    line=dict(color='#E74C3C', width=2.5),
    marker=dict(size=5, color='#C0392B'),
    name='Costo total',
    hovertemplate='Paso %{x}<br>Costo: %{y:.4f}<extra></extra>'
))

fig_cost.update_layout(
    title=dict(text='  Evolución del Costo durante el Entrenamiento', font_size=16),
    xaxis_title='Paso (iteración)',
    yaxis_title='Costo C',
    template='plotly_white',
    height=400,
    showlegend=True,
    annotations=[
        dict(x=0, y=costs[0], text=f'Inicio: {costs[0]:.3f}',
             showarrow=True, arrowhead=2, ax=40, ay=-30,
             font=dict(color='#E74C3C', size=12)),
        dict(x=steps[-1], y=costs[-1], text=f'Final: {costs[-1]:.4f}',
             showarrow=True, arrowhead=2, ax=-40, ay=-30,
             font=dict(color='#27AE60', size=12)),
    ]
)

fig_cost.show()
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

6.2 Trayectoria de los pesos en el espacio 3D

```
w1_hist = [h['w1'] for h in history]
w2_hist = [h['w2'] for h in history]
b_hist  = [h['b']  for h in history]

fig_3d = go.Figure()
```

```python
# Trayectoria de los pesos
fig_3d.add_trace(go.Scatter3d(
    x=w1_hist, y=w2_hist, z=costs,
    mode='lines+markers',
    line=dict(color=costs, colorscale='Viridis', width=5),
    marker=dict(
        size=4,
        color=costs,
        colorscale='Viridis',
        showscale=True,
        colorbar=dict(title='Costo'),
    ),
    name='Trayectoria',
    hovertemplate='w1=%{x:.3f}<br>w2=%{y:.3f}<br>Costo=%{z:.4f}<extra></extra>'
))

# Punto inicial
fig_3d.add_trace(go.Scatter3d(
    x=[w1_hist[0]], y=[w2_hist[0]], z=[costs[0]],
    mode='markers+text',
    marker=dict(size=10, color='red', symbol='circle'),
    text=['Inicio'],
    textposition='top center',
    name='Inicio'
))

# Punto final
fig_3d.add_trace(go.Scatter3d(
    x=[w1_hist[-1]], y=[w2_hist[-1]], z=[costs[-1]],
    mode='markers+text',
    marker=dict(size=10, color='green', symbol='diamond'),
    text=['Final'],
    textposition='top center',
    name='Final'
))

fig_3d.update_layout(
    title=dict(text='  Trayectoria de los Pesos en el Espacio de la Función de Costo',
               font_size=15),
    scene=dict(
        xaxis_title='w ',
        yaxis_title='w ',
        zaxis_title='Costo C',
        camera=dict(eye=dict(x=1.5, y=1.5, z=1.2))
    ),
    height=550,
    template='plotly_white'
```

```
)

fig_3d.show()
```

```
Unable to display output for mime type(s): text/html
```

6.3 Mapa de contorno de la función de costo (w1 vs w2)

```
# Calcular la superficie de costo sobre una malla de w1, w2
# (manteniendo b fijo en su valor final para la visualización)
b_fixed = np.mean(b_hist)

w1_range = np.linspace(-4, 5, 120)
w2_range = np.linspace(-1, 5, 120)
W1_grid, W2_grid = np.meshgrid(w1_range, w2_range)

C_grid = np.zeros_like(W1_grid)
for i in range(W1_grid.shape[0]):
    for j in range(W1_grid.shape[1]):
        _, a_tmp = forward_pass(X, b_fixed, W1_grid[i, j], W2_grid[i, j])
        C_grid[i, j] = compute_cost(a_tmp, y)

fig_contour = go.Figure()

# Mapa de contorno
fig_contour.add_trace(go.Contour(
    x=w1_range, y=w2_range, z=C_grid,
    colorscale='RdYlGn_r',
    contours=dict(
        showlabels=True,
        labelfont=dict(size=10, color='black')
    ),
    colorbar=dict(title='Costo'),
    name='Superficie de costo',
    hovertemplate='w1=%{x:.2f}<br>w2=%{y:.2f}<br>Costo=%{z:.4f}<extra></extra>'
))

# Trayectoria de los pesos
fig_contour.add_trace(go.Scatter(
    x=w1_hist, y=w2_hist,
    mode='lines+markers',
    line=dict(color='white', width=2),
    marker=dict(
        size=6,
        color=list(range(len(w1_hist))),
        colorscale='Blues',
        showscale=False,
```

```
    ),
    name='Trayectoria de pesos',
    hovertemplate='Paso %{text}<br>w1=%{x:.3f}<br>w2=%{y:.3f}<extra></extra>',
    text=[str(h['step']) for h in history]
))

# Inicio y fin
fig_contour.add_trace(go.Scatter(
    x=[w1_hist[0]], y=[w2_hist[0]],
    mode='markers+text',
    marker=dict(size=14, color='red', symbol='star'),
    text=['Inicio'], textposition='top right',
    name='Inicio', textfont=dict(color='white', size=12)
))
fig_contour.add_trace(go.Scatter(
    x=[w1_hist[-1]], y=[w2_hist[-1]],
    mode='markers+text',
    marker=dict(size=14, color='lime', symbol='star'),
    text=['Mínimo'], textposition='top right',
    name='Mínimo encontrado', textfont=dict(color='white', size=12)
))

# Flechas de gradiente (cada 5 pasos)
for idx in range(0, len(history)-1, 5):
    h = history[idx]
    z_tmp, a_tmp = forward_pass(X, h['b'], h['w1'], h['w2'])
    _, gw1, gw2 = compute_gradients(X, y, z_tmp, a_tmp)
    scale = 0.4
    fig_contour.add_annotation(
        x=h['w1'] - scale * gw1,
        y=h['w2'] - scale * gw2,
        ax=h['w1'], ay=h['w2'],
        xref='x', yref='y', axref='x', ayref='y',
        showarrow=True, arrowhead=3,
        arrowsize=1.2, arrowwidth=1.5,
        arrowcolor='cyan'
    )

fig_contour.update_layout(
    title=dict(
        text='  Mapa de Contorno del Costo y Trayectoria del Descenso por Gradiente',
        font_size=15
    ),
    xaxis_title='w  (Peso 1)',
    yaxis_title='w  (Peso 2)',
    template='plotly_dark',
    height=600,
```

```
    legend=dict(
        bgcolor='rgba(0,0,0,0.5)',
        font=dict(color='white')
    )
)

fig_contour.show()
```

```
Unable to display output for mime type(s): text/html
```

6.4 Paso a paso interactivo (detalle de cada iteración)

```
# Tabla detallada de los primeros 16 pasos (como en las imágenes)
print("                              ")
print("  Paso      b         w         w        Costo    ")
print("                              ")
for h in history[:17]:
    print(f"  {h['step']:>4}   {h['b']:>8.3f}   {h['w1']:>8.3f}   {h['w2']:>8.3f}   {h['cost']:>8
print("                              ")
```

```
  Paso      b         w         w        Costo

     0     1.000    -2.000     3.000    1.5853
     1    -0.007    -2.202     2.976    1.2632
     2    -0.552    -1.902     3.098    1.1184
     3    -0.928    -1.543     3.175    1.0137
     4    -1.268    -1.204     3.150    0.9208
     5    -1.606    -0.856     3.028    0.8144
     6    -1.963    -0.466     2.816    0.6769
     7    -2.307    -0.009     2.568    0.5266
     8    -2.537     0.492     2.412    0.4057
     9    -2.709     0.929     2.323    0.3235
    10    -2.877     1.267     2.263    0.2711
    11    -3.021     1.537     2.248    0.2371
    12    -3.167     1.742     2.251    0.2136
    13    -3.296     1.913     2.282    0.1960
    14    -3.426     2.049     2.319    0.1820
    15    -3.542     2.170     2.370    0.1702
    16    -3.657     2.272     2.421    0.1600
```

6.5 Evolución de los pesos individuales

```
fig_weights = make_subplots(
    rows=1, cols=3,
    subplot_titles=('Sesgo b', 'Peso w ', 'Peso w '),
```

```
    shared_yaxes=False
)

colors = ['#9B59B6', '#3498DB', '#E67E22']
params = [b_hist, w1_hist, w2_hist]
labels = ['b', 'w ', 'w ']

for col, (param, label, color) in enumerate(zip(params, labels, colors), 1):
    fig_weights.add_trace(
        go.Scatter(
            x=steps, y=param,
            mode='lines+markers',
            name=label,
            line=dict(color=color, width=2),
            marker=dict(size=4),
            hovertemplate=f'Paso %{{x}}<br>{label}=%{{y:.4f}}<extra></extra>'
        ),
        row=1, col=col
    )
    # Línea del valor final
    fig_weights.add_hline(
        y=param[-1], line_dash='dash',
        line_color=color, opacity=0.5,
        row=1, col=col
    )

fig_weights.update_layout(
    title=dict(text='  Evolución de los Parámetros durante el Entrenamiento', font_size=15),
    template='plotly_white',
    height=380,
    showlegend=False
)
fig_weights.update_yaxes(title_text='Valor')
fig_weights.update_xaxes(title_text='Paso')

fig_weights.show()
```

```
Unable to display output for mime type(s): text/html
```

6.6 Superficie de activación aprendida

```
# Visualizar qué aprendió la red: la función que mapea (x1,x2) → ŷ
x_grid = np.linspace(-0.1, 1.1, 80)
y_grid = np.linspace(-0.1, 1.1, 80)
X1g, X2g = np.meshgrid(x_grid, y_grid)

X_flat = np.column_stack([X1g.ravel(), X2g.ravel()])
```

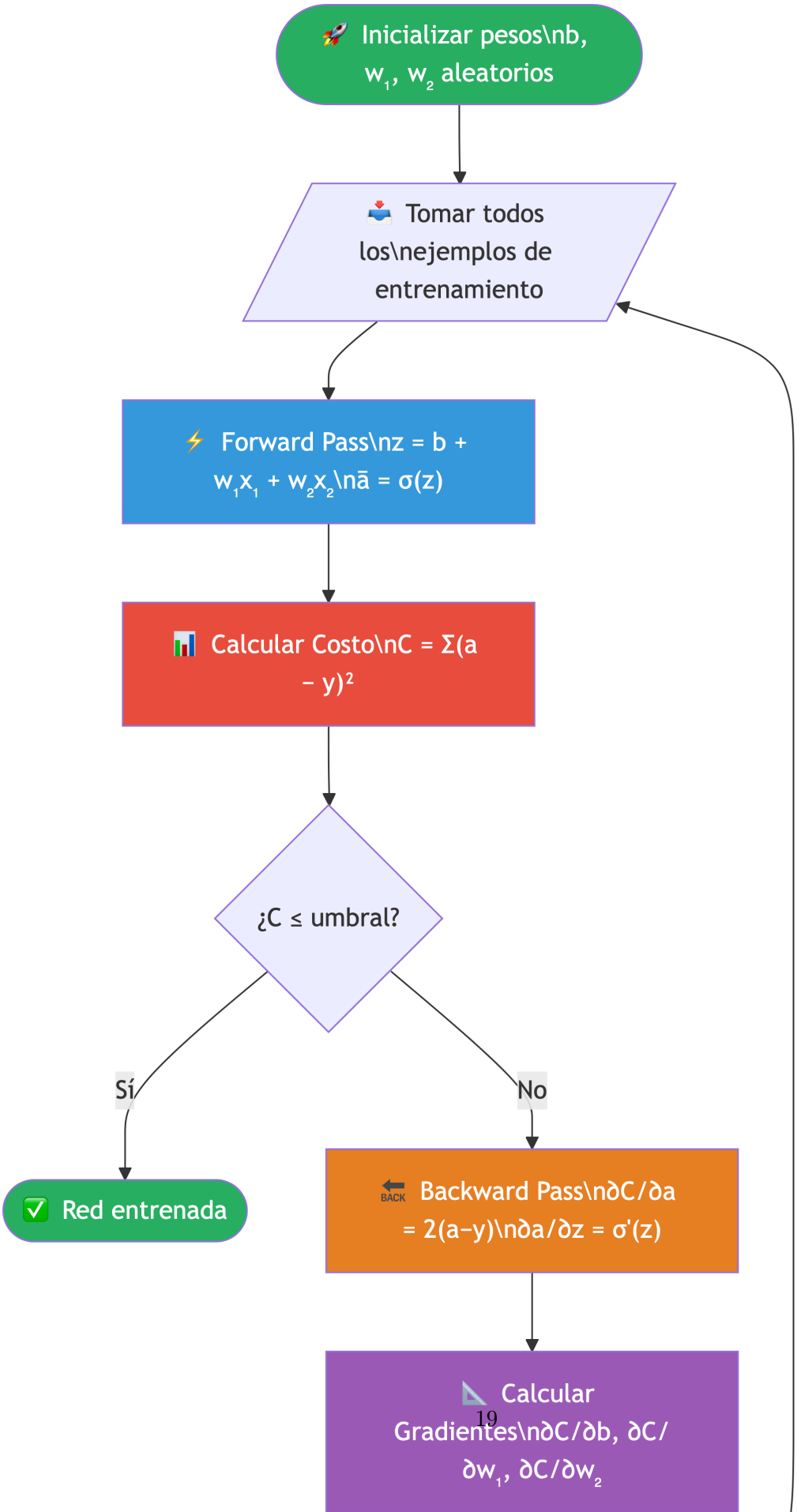
```
_, A_flat = forward_pass(X_flat, b_f, w1_f, w2_f)
A_grid = A_flat.reshape(X1g.shape)

fig_surface = go.Figure()

fig_surface.add_trace(go.Surface(
    x=x_grid, y=y_grid, z=A_grid,
    colorscale='Blues',
    opacity=0.85,
    name='Función aprendida',
    showscale=True,
    colorbar=dict(title='ŷ (probabilidad)')
))

# Puntos de entrenamiento
colors_pts = ['red' if yi == 0 else 'lime' for yi in y]
for i, (xi, yi) in enumerate(zip(X, y)):
    fig_surface.add_trace(go.Scatter3d(
        x=[xi[0]], y=[xi[1]], z=[float(yi)],
        mode='markers+text',
        marker=dict(size=8, color=colors_pts[i],
                    symbol='circle', line=dict(color='black', width=1)),
        text=[f'({int(xi[0])},{int(xi[1])})→{int(yi)}'],
        textposition='top center',
        showlegend=False
    ))

fig_surface.update_layout(
    title=dict(text='  Función Aprendida por la Red Neuronal (AND Gate)', font_size=15),
    scene=dict(
        xaxis_title='x ',
        yaxis_title='x ',
        zaxis_title='ŷ (salida)',
        camera=dict(eye=dict(x=1.6, y=-1.6, z=1.2))
    ),
    height=550,
    template='plotly_white'
)

fig_surface.show()
```

```
Unable to display output for mime type(s): text/html
```

---

7. Flujo de Entrenamiento Completo

```
flowchart TD
    A([ Inicializar pesos\nb, w , w  aleatorios]) --> B

    B[/" Tomar todos los\nejemplos de entrenamiento"/]
    B --> C

    C[" Forward Pass\nz = b + w x  + w x \nā =  (z)"]
    C --> D

    D[" Calcular Costo\nC = Σ(a - y)²"]
    D --> E{¿C  umbral?}

    E -- Sí --> Z([ Red entrenada])

    E -- No --> F

    F[" Backward Pass\n C/ a = 2(a-y)\n a/ z =  '(z)"]
    F --> G

    G[" Calcular Gradientes\n C/ b,  C/ w ,  C/ w "]
    G --> H

    H[" Actualizar Pesos\nw ← w - k· C/ w\nb ← b - k· C/ b"]
    H --> B

    style A fill:#27AE60,color:white
    style Z fill:#27AE60,color:white
    style C fill:#3498DB,color:white
    style D fill:#E74C3C,color:white
    style F fill:#E67E22,color:white
    style G fill:#9B59B6,color:white
    style H fill:#1ABC9C,color:white
```


🚀 Inicializar pesos\nb, w_1, w_2 aleatorios
📩 Tomar todos los\nejemplos de entrenamiento
⚡ Forward Pass\nz = b + w_1x_1 + w_2x_2\nā = σ(z)
📊 Calcular Costo\nC = Σ(a − y)²
¿C ≤ umbral?
Sí
No
✅ Red entrenada
BACK Backward Pass\n∂C/∂a = 2(a−y)\n∂a/∂z = σ'(z)
📐 Calcular Gradientes\n∂C/∂b, ∂C/∂w_1, ∂C/∂w_2

---

8. Resumen Conceptual

¿Cómo aprende una red neuronal?

El proceso de aprendizaje sigue este ciclo:

1. **Inicialización**: Los pesos se inicializan (aleatoriamente o con valores específicos).
2. **Propagación hacia adelante** (*forward pass*): Para cada ejemplo de entrenamiento, calculamos la predicción de la red usando los pesos actuales.
3. **Cálculo del error**: Medimos qué tan lejos están las predicciones de los valores reales usando la función de costo.
4. **Propagación hacia atrás** (*backpropagation*): Calculamos el gradiente del costo respecto a cada peso usando la regla de la cadena.
5. **Actualización**: Ajustamos los pesos moviéndonos en la dirección opuesta al gradiente (cuesta abajo en la superficie de error).
6. **Repetición**: Repetimos los pasos 2-5 hasta que el error sea suficientemente pequeño.

Analogía geográfica

Imagina que estás en una montaña con niebla (no puedes ver el valle): - Tu **posición** = valores actuales de los pesos - Tu **altitud** = valor del costo - El **gradiente** = la pendiente bajo tus pies - El **Descenso por Gradiente** = dar pasos pequeños cuesta abajo

Con suficientes pasos, llegarás al valle (mínimo del costo).

```
#   Resumen final
print("=" * 55)
print("  RESUMEN DEL ENTRENAMIENTO")
print("=" * 55)
print(f"  Problema          : Compuerta AND (2 entradas)")
print(f"  Activación        : Sigmoide  (z)")
print(f"  Función de pérdida: Error Cuadrático")
print(f"  Tasa de aprendizaje: {learning_rate}")
print(f"  Pasos de entrenamiento: {n_steps}")
print("-" * 55)
print(f"  Costo inicial     : {costs[0]:.4f}")
print(f"  Costo final       : {costs[-1]:.6f}")
print(f"  Reducción         : {(1 - costs[-1]/costs[0])*100:.1f}%")
print("-" * 55)
print(f"  Parámetros finales:")
print(f"    b  = {final['b']:>8.4f}")
print(f"    w  = {final['w1']:>8.4f}")
print(f"    w  = {final['w2']:>8.4f}")
print("=" * 55)
```

```
=========================================================
  RESUMEN DEL ENTRENAMIENTO
=========================================================
  Problema          : Compuerta AND (2 entradas)
  Activación        : Sigmoide  (z)
  Función de pérdida: Error Cuadrático
  Tasa de aprendizaje: 10
  Pasos de entrenamiento: 50
---------------------------------------------------------
  Costo inicial     : 1.5853
  Costo final       : 0.048993
  Reducción         : 96.9%
---------------------------------------------------------
  Parámetros finales:
    b  =  -5.7867
    w  =   3.7875
    w  =   3.7879
=========================================================
```

9. Ejemplo adicional: grafo computacional con bifurcaciones

En nuestra red neuronal simple (Sección 3), el grafo computacional era una **cadena lineal**: cada nodo alimenta exactamente al siguiente. Pero en funciones más complejas — y en redes neuronales reales — los grafos **se bifurcan**: una misma variable intermedia puede alimentar varias operaciones a la vez. Esto cambia la forma en que se calculan los gradientes durante la retropropagación.

Para comprender este patrón, trabajaremos un ejemplo tomado de Deisenroth *et al.* (2020, Ejemplo 5.14).

9.1 La función y sus variables intermedias

Consideremos la función:

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos(x^2 + \exp(x^2))$$

Calcular $\partial f/\partial x$ directamente sobre esta expresión sería largo y propenso a errores. La idea clave es que **si implementáramos f en un computador**, no evaluaríamos toda la expresión de golpe, sino que guardaríamos resultados parciales en **variables intermedias**:

$$a = x^2, \qquad b = \exp(a), \qquad c = a + b, \qquad d = \sqrt{c}, \qquad e = \cos(c), \qquad f = d + e$$

Cada variable almacena el resultado de una sola operación elemental ($(\cdot)^2$, exp, $+$, $\sqrt{\cdot}$, cos). Esto es exactamente lo que hacen las librerías de redes neuronales como PyTorch o TensorFlow internamente.

9.2 El grafo computacional (Forward Pass)

El conjunto de ecuaciones anteriores se puede representar como un **grafo computacional** (Figure 2): los **nodos naranjas** son variables (valores que se almacenan) y los **nodos azules** son operaciones (funciones elementales). Las **flechas sólidas** muestran el flujo de datos de izquierda a derecha — el *forward pass*.

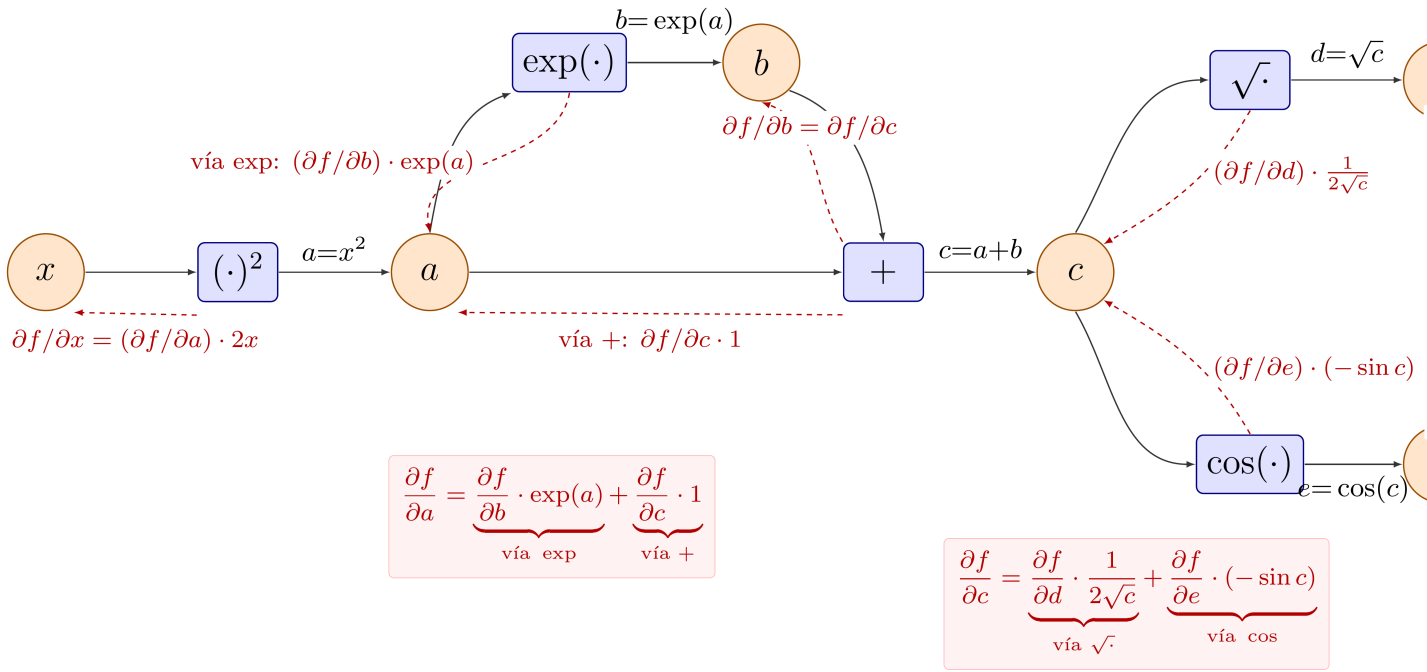


Figure 2: Grafo computacional del Ejemplo 5.14 (Deisenroth *et al.*, 2020): las flechas sólidas representan el forward pass y las flechas rojas discontinuas el backward pass (retropropagación).

Observa dos características importantes del grafo:

1. **Bifurcación en a:** la variable a alimenta dos operaciones distintas — $\exp(\cdot)$ (rama superior) y $+$ (directamente). Esto refleja que $a = x^2$ aparece en **dos lugares** de la expresión original.
2. **Bifurcación en c:** la variable c también se divide en dos ramas — $\sqrt{\cdot}$ (hacia d) y $\cos(\cdot)$ (hacia e) — que luego se reúnen en la suma final $f = d + e$.

9.3 Derivadas locales de cada nodo

Antes de retropropagar, necesitamos las derivadas de cada operación elemental respecto a su entrada directa. Son las derivadas que aprendemos en cálculo básico:

Ecuación	Derivada local	Valor
$a = x^2$	$\partial a/\partial x$	$2x$
$b = \exp(a)$	$\partial b/\partial a$	$\exp(a)$
$c = a + b$	$\partial c/\partial a = \partial c/\partial b$	1
$d = \sqrt{c}$	$\partial d/\partial c$	$\frac{1}{2\sqrt{c}}$
$e = \cos(c)$	$\partial e/\partial c$	$-\sin(c)$

Ecuación	Derivada local	Valor
$f = d + e$	$\partial f/\partial d = \partial f/\partial e$	1

> **Observación:** Cada derivada local es muy sencilla. La potencia del grafo computacional es que **nunca necesitamos derivar la función completa** — solo combinamos estas piezas simples.

9.4 Retropropagación paso a paso (Backward Pass)

Ahora recorremos el grafo **de derecha a izquierda** (flechas rojas discontinuas en Figure 2), aplicando la regla de la cadena en cada nodo.

Paso 1 — Nodo f (la salida):

Comenzamos con la derivada de f respecto a sí misma:

$$\frac{\partial f}{\partial f} = 1$$

Paso 2 — Nodo + final ($f = d + e$):

Como la suma tiene derivada 1 respecto a cada sumando, el gradiente se **copia** idéntico a ambas ramas:

$$\frac{\partial f}{\partial d} = 1, \qquad \frac{\partial f}{\partial e} = 1$$

Paso 3 — Nodo $\sqrt{\cdot}$ ($d = \sqrt{c}$):

Aplicamos la regla de la cadena a lo largo de la rama superior. Multiplicamos el gradiente que llega ($\partial f/\partial d = 1$) por la derivada local:

$$\left.\frac{\partial f}{\partial c}\right|_{\text{vía } \sqrt{\cdot}} = \frac{\partial f}{\partial d} \cdot \frac{\partial d}{\partial c} = 1 \cdot \frac{1}{2\sqrt{c}}$$

Paso 4 — Nodo $\cos(\cdot)$ ($e = \cos(c)$):

Lo mismo por la rama inferior:

$$\left.\frac{\partial f}{\partial c}\right|_{\text{vía } \cos} = \frac{\partial f}{\partial e} \cdot \frac{\partial e}{\partial c} = 1 \cdot (-\sin c)$$

Paso 5 — Combinación en c (primera bifurcación):

El nodo c influye en f por **dos caminos** (vía $\sqrt{\cdot}$ y vía cos). Su gradiente total es la **suma** de ambas contribuciones:

$$\boxed{\frac{\partial f}{\partial c} = \frac{1}{2\sqrt{c}} + (-\sin c) = \frac{1}{2\sqrt{c}} - \sin(c)}$$

> **¿Por qué se suman?** Si mueves c un poquito, el efecto se propaga por los dos caminos *simultáneamente*. El efecto total es la suma de ambos efectos parciales. Esto es la **derivada total** del cálculo multivariable.

Paso 6 — Nodo + ($c = a + b$):

De nuevo, la suma copia el gradiente a ambas entradas:

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c} \cdot \frac{\partial c}{\partial b} = \frac{\partial f}{\partial c} \cdot 1 = \frac{\partial f}{\partial c}$$

Paso 7 — Nodo $\exp(\cdot)$ **($b = \exp(a)$):**

$$\left.\frac{\partial f}{\partial a}\right|_{\text{vía exp}} = \frac{\partial f}{\partial b} \cdot \frac{\partial b}{\partial a} = \frac{\partial f}{\partial c} \cdot \exp(a)$$

Paso 8 — Combinación en a (segunda bifurcación):

El nodo a también alimenta dos caminos: uno a través de $\exp(\cdot)$ y otro directamente al $+$. Sumamos:

$$\boxed{\frac{\partial f}{\partial a} = \underbrace{\frac{\partial f}{\partial b} \cdot \exp(a)}_{\text{vía exp}} + \underbrace{\frac{\partial f}{\partial c} \cdot 1}_{\text{vía } +}}$$

Paso 9 — Nodo $(\cdot)^2$ **($a = x^2$):**

Finalmente, multiplicamos por la derivada local $2x$:

$$\boxed{\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \cdot 2x}$$

9.5 La eficiencia del grafo computacional

Un resultado notable es que **el cómputo de la derivada tiene un costo similar al cómputo de la función misma**. Esto es sorprendente porque la expresión analítica de $\partial f/\partial x$ (si la expandiéramos completamente) es mucho más larga que la de $f(x)$.

El truco es que nunca expandimos la derivada completa. En lugar de eso, tratamos cada $\partial f/\partial a$, $\partial f/\partial c$, etc. como una **variable numérica** que calculamos y reutilizamos. Esto es exactamente lo que hace el algoritmo de retropropagación:

1. **Forward pass:** calcula y almacena cada variable intermedia (a, b, c, d, e, f).
2. **Backward pass:** recorre el grafo al revés, calculando un gradiente por nodo, reutilizando los valores almacenados.

9.6 Resumen: las dos reglas del grafo

Situación en el grafo	Qué hacer con los gradientes	Ejemplo en este grafo
Un camino (A $\to$ op $\to$ B)	**Multiplicar** por la derivada local	$\partial f/\partial x = (\partial f/\partial a) \cdot 2x$

Situación en el grafo	Qué hacer con los gradientes	Ejemplo en este grafo
Bifurcación (A $\to$ B y A $\to$ C)	**Sumar** las contribuciones de cada camino	$\partial f/\partial c = \frac{1}{2\sqrt{c}} - \sin(c)$

Conexión con nuestra red neuronal: En la Sección 3, el grafo era una cadena sin bifurcaciones, así que solo necesitábamos multiplicar. En este ejemplo, los nodos a y c se bifurcan, y por eso aparecen sumas. En redes neuronales con múltiples capas y conexiones residuales (como ResNet), las bifurcaciones son muy frecuentes, y esta misma regla se aplica en cada una.

Referencias y Lecturas Adicionales

- Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.

Documento generado con Quarto · Código Python · Visualizaciones Plotly